

HPCG2 - Reto 1

Solucion de la ecuacion de Poisson 1D con Jacobi Iterativo (Serial, Threads y Procesos)

Integrantes

- David Santiago Lugo Cabrera
 - Jose Aroudo Junior de Asis Pinedo
 - Angie Carolina Vargas Villegas
-

1. Analisis del algoritmo de Jacobi Iterativo para Poisson 1D

Se considera el problema de frontera:

- $-u''(x) = f(x), x \in (0,1)$
- $u(0)=0, u(1)=0$

Con discretizacion por diferencias finitas de segundo orden sobre N nodos internos:

- $h = 1/(N+1)$
- $-u_{i-1} + 2u_i - u_{i+1} = h^2 f_i, i=1..N$

El metodo de Jacobi obtiene una sucesion $u^{(k)}$:

$$u_i^{(k+1)} = 0.5 * (u_{i-1}^{(k)} + u_{i+1}^{(k)} + h^2 f_i)$$

1.1 Costo computacional

- Costo por iteracion: $O(N)$.
- Costo total: $O(N * K)$ donde K depende de tolerancia y condicionamiento.
- Memoria: $O(N)$ usando dos vectores (u y u_{next}).

1.2 Criterio de convergencia

Se usa norma infinito sobre el cambio iterativo:

$$residual = \max_i |u_i^{(k+1)} - u_i^{(k)}|$$

El algoritmo termina cuando $residual < tol$ o al llegar a max_iter .

1.3 Dependencias de datos

- Cada $u_i^{(k+1)}$ depende solo de vecinos en iteracion previa (k).
- Dentro de la misma iteracion no hay dependencia de escritura entre nodos internos.
- Esto habilita paralelismo por particion de dominio (bloques de indices).

2. Implementacion serial

Se implemento en C en `src/serial/jacobi_serial.c` con:

- Vectores dinamicos para `u`, `u_next` y `f`.
- Intercambio de punteros por iteracion para evitar copia completa.
- Medicion de tiempo con `clock_gettime(CLOCK_MONOTONIC)`.

Resumen de flujo:

1. Inicializar `u=0` y `f`.
2. Repetir actualizacion Jacobi sobre nodos `1..N`.
3. Calcular `residual` maximo de la iteracion.
4. Intercambiar `u <-> u_next`.
5. Verificar convergencia.

3. Analisis de concurrencia y optimizacion (Threads y fork)

3.1 Threads (`pthread`)

Archivo: `src/threads/jacobi_threads.c`

Estrategia:

- Particion de nodos en `T` bloques contiguos.
- Cada hilo actualiza su rango y calcula un maximo local.
- Barrera 1: esperar fin de calculo por iteracion.
- Hilo 0 reduce maximos, decide convergencia e intercambia punteros.
- Barrera 2: publicar estado para la siguiente iteracion.

Ventajas:

- Menor sobrecarga de comunicacion que procesos.
- Memoria compartida natural.
- Mejor latencia de sincronizacion para granularidades finas.

Riesgos:

- Sobresincronizacion si `N` es pequeno.
- Falsa comparticion en estructuras compartidas.
- Escalamiento limitado por ancho de banda de memoria.

3.2 Procesos (`fork` + memoria compartida)

Archivo: `src/processes/jacobi_processes.c`

Estrategia:

- `fork` de procesos trabajadores persistentes.
- Vectores alojados en memoria compartida (`mmap MAP_SHARED`).
- Barreras `pthread_barrier` con atributo `PTHREAD_PROCESS_SHARED`.
- El proceso padre reduce maximos locales, intercambia punteros y decide parada.

Ventajas:

- Aislamiento de espacio de direcciones.
- Modelo valido para escenarios de multiproceso y afinidad por CPU.

Riesgos:

- Overhead de sincronizacion y gestion de procesos mayor que en hilos.
- Mayor sensibilidad a costos de IPC/sincronizacion.
- Complejidad de implementacion y depuracion superior.

3.3 Optimizacion de CPU y memoria

Optimizaciones aplicables:

- Compilacion con `-O3`.
- Particion balanceada para evitar desbalance de carga.
- Evitar copias completas del vector (swap de punteros).
- Acceso secuencial a memoria para mejorar localidad espacial.
- Fijar afinidad CPU (opcional) para reducir migraciones de hilos/procesos.
- Ajustar numero de workers a nucleos fisicos disponibles.

4. Analisis de causas de desempeno

4.1 Comportamiento esperado

- Serial: referencia base.
- Threads: mejor speedup en tamanos de problema medios/grandes.
- Procesos: mejora posible, pero usualmente menor que threads por overhead.

4.2 Causas comunes de degradacion

- Costo de barreras por iteracion (especialmente con `N` bajo).
- Limite de ancho de banda de memoria en muchos workers.
- Desbalance por particiones no uniformes.
- Competencia de cache y falsa comparticion.
- Variabilidad del sistema operativo (scheduler, carga externa).

4.3 Metodologia de medicion ejecutada

1. Se uso el mismo problema para todas las pruebas: `N=200000`, `max_iter=2000`, `tol=1e-20`.
2. Se ejecutaron `10` corridas por configuracion.
3. Se midio `Tiempo_s` reportado por cada binario.
4. Se calculo promedio, desviacion estandar y mejor tiempo.
5. Se calculo speedup: `T_serial / T_paralelo` y eficiencia: `speedup / workers`.

Entorno de prueba:

- CPU: AMD Ryzen 5 5600X (6 nucleos fisicos, 12 logicos).
- SO: Linux (Ubuntu 24.04).
- Compilacion: `-O3` con `-D_POSIX_C_SOURCE=200809L` (y `-pthread` en versiones paralelas).

4.4 Tabla de resultados

Metodo	Workers	N	Tiempo promedio (s)	Desv. estandar (s)	Mejor tiempo (s)	Speedup	Eficiencia
Serial	1	200000	0.246254	0.009842	0.238079	1.0000	1.0000
Threads	2	200000	0.191951	0.002321	0.188957	1.2829	0.6415
Threads	4	200000	0.114501	0.003423	0.109368	2.1507	0.5377
Threads	8	200000	0.093268	0.003541	0.086090	2.6403	0.3300
Procesos	2	200000	0.224581	0.005514	0.216482	1.0965	0.5483
Procesos	4	200000	0.135012	0.004457	0.126643	1.8239	0.4560
Procesos	8	200000	0.108352	0.006842	0.096207	2.2727	0.2841

Nota: la version `fork` no se pudo ejecutar en Windows nativo por dependencia de APIs POSIX (`mmap`, `fork`, barreras de proceso compartido). Para medir procesos, se requiere Linux o WSL con distribucion instalada. En este caso, se hicieron todas las pruebas en linux(distribución ubuntu).

5. Conclusiones

- Las pruebas validan que Jacobi 1D se beneficia de paralelizacion por particion de dominio, ya que cada punto interno se actualiza con datos de la iteracion anterior.
- La implementacion con `pthread` mejora frente a serial en todos los casos medidos, con speedup de **1.2829x** (2 hilos), **2.1507x** (4 hilos) y **2.6403x** (8 hilos).
- La implementacion con procesos (`fork`) tambien mejora frente a serial, con speedup de **1.0965x** (2 procesos), **1.8239x** (4 procesos) y **2.2727x** (8 procesos).
- Para el mismo numero de workers, `pthread` supera consistentemente a `fork`, lo cual es coherente con menor overhead de sincronizacion y comunicacion en memoria compartida intra-proceso.
- La eficiencia cae al aumentar workers en ambos enfoques (threads: **0.6415** -> **0.5377** -> **0.3300**; procesos: **0.5483** -> **0.4560** -> **0.2841**), evidenciando costos de barrera y limites de ancho de banda de memoria.
- En este experimento, `threads` con 8 workers ofrece el mejor tiempo promedio (**0.093268 s**) y el mejor speedup global (**2.6403x**).